Practical-7: (Vue JS**)**
Demonstrate how to create a custom directive and how to use it through HTML web pages. **(Note: Students need to implement the practical based on the project title selected in Practical 1.5.)**:

I. Create a custom directive to transform text to uppercase when clicked.
II. Creating a Dynamic List with Vue.js Custom Directive
III. Create a directive that formats and displays dates in a human-readable format

Step-1: Define the Directive: Use `Vue.directive()` to define your custom directive. You need to specify a name for the directive and an object containing hook functions that define the directive's behavior.

Step-2: Hook Functions: There are several hook functions you can use to define the behavior of your directive. The most commonly used ones are:

- **`bind`**: Called only once when the directive is first bound to the element. You can perform one-time setup here.
- **`inserted`**: Called when the bound element is inserted into the DOM. This is where you can perform DOM manipulation.
- **`update`**: Called when the bound element's value or expression changes.
- **`componentUpdated`**: Called after the component containing the bound element has been updated.
- **`unbind`**: Called only once when the directive is unbound from the element.

Step-3: Accessing the Element and Data**: Within the hook functions, you can access the element the directive is bound to, as well as any data passed to the directive.**

I. Create a custom directive to transform text to uppercase when clicked.

File: capitalize.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Custom Directive Example</title>
  <script src="https://cdn.jsdelivr.net/npm/vue@2"></script>
</head>
<body>

<div id="app">
  <p v-uppercase>Hello, this text will be uppercased when clicked!</p>
</div>

<script src="app.js"></script>
</body>
</html>
```

`<p v-uppercase>`: This is a paragraph element (`<p>`) to which a Vue directive named `v-uppercase` is applied. Directives in Vue.js are special attributes prefixed with `v-`, which allow you

to apply reactive behavior to the DOM. In this case, `v-uppercase` is a custom directive that will be triggered when the paragraph is interacted with.

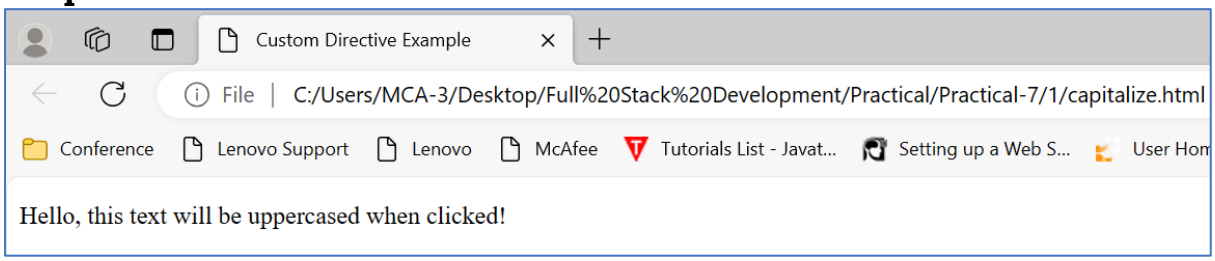
File: app.js

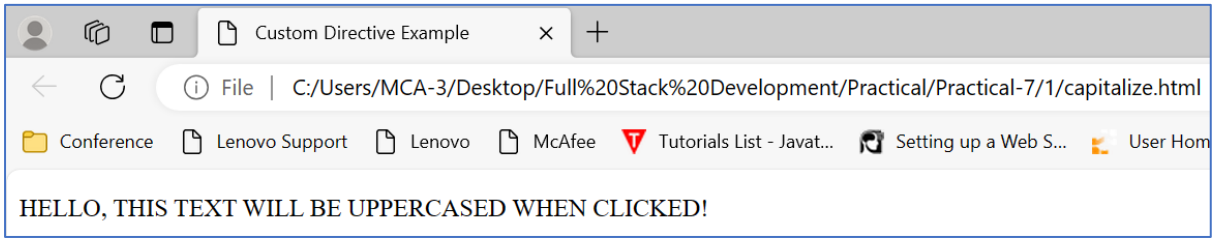
```
Vue.directive('uppercase', {
  bind(el) {
    el.style.cursor = 'pointer'; // Changes the cursor style to pointer when hovering over the element

    el.addEventListener('click', () => { // Attaches a click event listener to the element
      const text = el.innerText; // Retrieves the text content of the element
      el.innerText = text.toUpperCase(); // Changes the text content to uppercase
    });
  }
});

// Create a new Vue instance
new Vue({
  el: '#app' // Mounts the Vue instance to the element with the ID 'app'
});
```

Output:





Here's what each part of the code does:

1. `Vue.directive('uppercase', { ... })`: This line registers a custom directive named `'uppercase'` with Vue.js. Custom directives in Vue.js allow you to extend the behavior of HTML elements. In this case, the directive will make the text content of an element uppercase when clicked.
2. `bind(el) { ... }`: This is the definition of the directive's **hook function**. The `bind` hook is called only once when the directive is first bound to the element. It receives the element (`el`) as its argument.

3. `el.style.cursor = 'pointer';`: This line sets the cursor style of the element to `pointer`, making it appear as a hand icon when hovered over. This is a visual cue to the user that the element is clickable.
4. `el.addEventListener('click', () => { ... });:` This line attaches a click event listener to the element. When the element is clicked, the callback function inside the event listener will be executed.
5. `const text = el.innerText;:` This line retrieves the text content of the element and stores it in the `text` variable.
6. `el.innerText = text.toUpperCase();:` This line sets the text content of the element to its uppercase version. It uses the `toUpperCase()` method to convert the text to uppercase.
7. `new Vue({ el: '#app' });:` This code creates a new Vue instance and mounts it to the element with the ID `app`. The Vue instance is responsible for managing the behavior and data of the application within the specified element.

Overall, this code defines a custom Vue directive named `'uppercase'` that makes the text content of an element uppercase when clicked. It also changes the cursor style of the element to indicate that it is clickable.

II. Creating a Dynamic List with Vue.js Custom Directive

File: index.htlm

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Custom Directive Example</title>
  <script src="https://cdn.jsdelivr.net/npm/vue@2"></script>
</head>
<body>

<div id="app">
  <ul v-list="items"></ul>
</div>

<script src="app.js"></script>
</body>
</html>
```

File: app.js

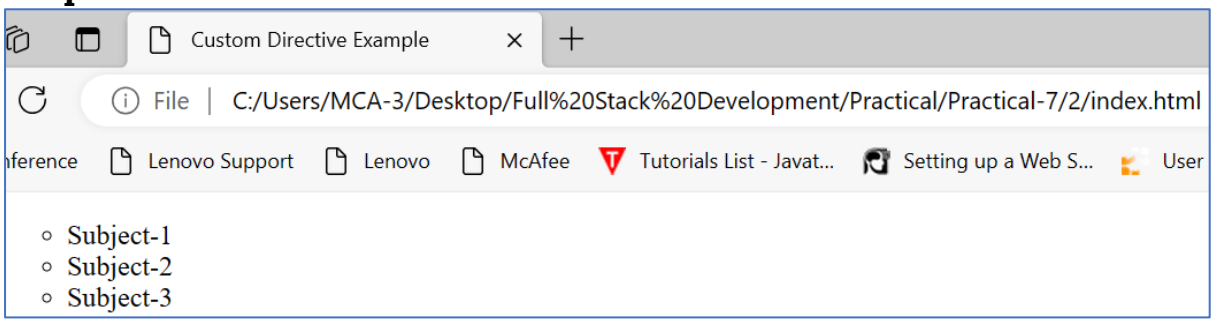
```
Vue.directive('list', {
  bind(el, binding) {
    const ul = document.createElement('ul'); // Create a new <ul> element

    binding.value.forEach(item => { // Iterate over each item in the binding value
      const li = document.createElement('li'); // Create a new <li> element
```

```
    li.textContent = item; // Set the text content of the <li> element to the current
        item
    ul.appendChild(li); // Append the <li> element to the <ul> element
  });
  el.appendChild(ul); // Append the <ul> element to the bound element (the element the
      directive is applied to)
 }
});

// Create a new Vue instance
new Vue({
 el: '#app', // Mount the Vue instance to the element with the ID 'app'
 data: {
  items: ['Subject-1', 'Subject-2', 'Subject-3'] // Sample list of items
 }
});
```

Output:



III. Create a directive that formats and displays dates in a human-readable format

File: v-format.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Custom Directive for Date Formatting</title>
  <script src="https://cdn.jsdelivr.net/npm/vue@2"></script>
</head>
<body>

<div id="app">
  <p v-format-date="date"> <!-- Apply the custom directive -->
    <!-- The content will be manipulated by the custom directive -->
  </p>
</div>

<script>
// Define the custom directive named 'format-date'
Vue.directive('format-date', {
```

```
  // When the bound element is inserted into the DOM...
  inserted: function(el, binding) {
    // Get the date from the binding value
    const date = binding.value;

    // Format the date into a human-readable format
    const formattedDate = formatDate(date);

    // Update the element's content with the formatted date
    el.textContent = formattedDate;
  }
});

// Create a new Vue instance
new Vue({
  el: '#app',
  data: {
    date: '2022-03-05T12:00:00' // Sample date
  }
});

// Helper function to format the date
function formatDate(dateString) {
  const options = { year: 'numeric', month: 'long', day: 'numeric' };
  return new Date(dateString).toLocaleDateString('en-US', options);
}
</script>

</body>
</html>
```

This line converts the input `dateString` into a JavaScript `Date` object using `new Date(dateString)`. Then, it uses the `toLocaleDateString` method to format the date based on the provided options (`options` object). The first argument `'en-US'` specifies the locale to be used for formatting the date.

Output:

